Design of a Vector Embedding for Capability Composition

PCCST503 Assignment 2
Technical report | 4 October 2026

1. Problem definition

This work answers PCCST503 Assignment 2 by designing a numerical representation of formally specified states, goals and executable capability specifications. The central question is whether the representation preserves the relationships required for compatibility and ordered composition. No planning, path search or replanning algorithm is developed.

An application is A=(S,C,S_I,G,R,K). A capability is C_i=(T_i,I_i,O_i,P_i,E_i,K_i,R_i,Q_i,Rel_i,A_i,M_i). The proposed solution is a transparent, finite-state operator embedding, called the Finite-State Capability Operator (FSCO) representation. It combines an exact transition vector with typed interface, mechanism, resource and quality blocks. It is designed here specifically for this assignment; it is not a pretrained language model.

Abstract: Across three synthetic problems, all 243 ordered capability-pair classifications agreed with a separate symbolic interpreter; all 288 composite-state checks and 2,592 atomic applicability checks agreed. Functional alternatives had cosine 1 but distinct full vectors. Matrix composition was associative and preserved goals, while vector addition did not represent sequential execution. These are correctness results in a closed universe, not predictive accuracy on unseen applications.

2. Design requirements

Identity: distinguish different modeled behavior and implementation. State awareness: explicitly represent each capability's applicability and successor state. Compatibility: distinguish directional state and data dependencies from symmetric resemblance. Composition: preserve order, overwritten values, and intermediate guards. Goal relevance: evaluate a specified transformer against a state and goal, including indirect contributors.

Operational properties: encode static quality and resource requirements, but check availability, resources and external inputs as hard runtime gates. Consistency: use stable schema ordering and reject comparisons between incompatible schemas. Efficiency: disclose state enumeration and dense storage costs. All conditions are formal JSON predicates rather than natural-language descriptions.

Assumptions: each variable has a declared finite domain; states are fully observed; effects are deterministic constant assignments; unspecified variables persist; predicates are conjunctions of eq, ne, gt, le or in. Port domains are finite sets. All outputs are retained until overwritten by another output with the same name. Resources are reusable presence requirements; resource_cost is an additive usage estimate, not a concurrent capacity constraint.

3. Related embedding approaches

Word2Vec [1] learns word vectors from context and motivates relation-preserving numerical representations. Its corpus statistics do not encode formal guard truth or assignment order. Sentence-BERT [2] produces

sentence vectors compared with cosine. This supports retrieval of similar descriptions, but descriptions are not the formal input here; symmetry of cosine cannot itself certify a directional dependency.

TransE [3] represents a relation as an approximate translation between entity vectors. That is attractive for compact relational structure, but a fixed displacement does not directly model guarded assignments: setting a Boolean true leaves an already-true state unchanged and changes a false state. Our design therefore uses state-dependent operators rather than a single state-independent displacement.

One-hot features are interpretable and distinguish a finite vocabulary but require explicit rules for composition. FSCO extends one-hot state features into a flattened partial-transition operator and augments it with role-separated interface and operational blocks. This offers exactness and auditability at the expense of compactness and generalization. The cosine threshold baseline is a simple similarity heuristic, not a trained Word2Vec, SBERT or TransE comparison.

4. Proposed representation

Enumerate the Cartesian product of all variable domains in sorted variable order: Omega={s_1,...,s_N}. Global policy violations yield zero transition rows. The state basis is shared by states, goal masks and capability operators within one application schema. Port signatures, atomic mechanisms and resources have sorted finite vocabularies built from the dataset.

For an atomic capability, form an N by N matrix F_i. An applicable source state has a single 1 in the column for its deterministic successor; an inapplicable state has an all-zero row. This captures negative conditions, numeric bounds, frame persistence and assignments without a learned approximation. flatten(F_i) is a vector in R^(N squared).

The full vector concatenates the transition block, input indicators, output indicators, mechanism indicators or counts, resource indicators, and seven operational values. Mechanism features encode the formal type and a canonical JSON mechanism. A composite sums mechanism counts and unions resources. Metadata retains interface declarations for readable diagnostics; the finite interface vocabulary also permits recovery of selected declarations from the indicator blocks.

5. Mathematical formulation

State: phi_S(s_k)=e_k in R^N, a row-basis vector. Goal: phi_G(G)=g in {0,1}^N where g[k]=1 iff s_k satisfies every goal condition. Thus phi_S(s) dot g is the exact goal truth value.

Let B_i(s) be the conjunction of P_i, K_i and the global policy K. Let f_i(s) apply E_i by overriding named variables. Define F_i[k,l]=1 iff B_i(s_k), K(f_i(s_k)) and f_i(s_k)=s_l; otherwise it is 0. Availability, resource presence and input values are intentionally outside this structural operator. Each row sum is 0 or 1.

Capability: phi_C(C_i)=[vec(F_i),u_I,u_O,h_M,h_R,q]. If L is the port vocabulary size, H the mechanism vocabulary size and J the resource vocabulary size, d_C=N^2+2L+H+J+7. The seven q entries are time_ms, resource_cost, money, risk, energy, reliability and availability. Raw units are preserved for accounting; use block selection rather than unscaled whole-vector cosine for functional judgments.

Functional similarity is sigma(i,j)=<vec(F_i),vec(F_j)>/(||F_i||_F ||F_j||_F), or 0 if a norm is zero. It is symmetric resemblance, not an execution test. State overlap can change this measure: identical effects with different guards need not score 1. The full-vector distance detects mechanism and quality differences; no capability-name-only coordinate is added.

For directional compatibility, a_i=F_i 1 and b_ij=F_i F_j 1. State coverage kappa(i,j)=sum(b_ij)/sum(a_i), with 0 for an empty denominator. An output matches an input iff names and type labels agree and the output domain is a subset of the input domain. D_ij=1 iff every required downstream input has such an upstream output.

Guaranteed structural compatibility is sum(a_i)>0 AND kappa=1 AND D_ij=1. Existential compatibility is sum(b_ij)>0 AND D_ij=1. These quantify over all modeled applicable source states, not only reachable states. The strict pair test requires upstream outputs to provide downstream inputs. A composition may instead expose missing inputs externally; the two interfaces answer different questions.

Goal relationship: V(C,s,G)=phi_S(s) F_C phi_G(G), in {0,1}. It tests structural success of a given capability or sequence; it is not a planner or a reliability estimate. Execute separately enforces operational gates. In the supplied chain, a capability's contribution is assessed by removing it from the fixed sequence and reevaluating V.

6. Capability composition model

compose([C_1,C_2,...,C_m]) executes left to right, representing C_m o ... o C_1. Decode each transition block, multiply F_comp=F_1 F_2 ... F_m, and flatten the result. This is a bilinear composition operation on the matrix blocks of two encoded vectors, rather than ordinary vector addition.

Proof of exactness: an applicable first row is e_l for its unique successor s_l. Multiplying that row by F_2 selects the unique successor row of s_l. If either step is inapplicable the resulting row is zero. Induction extends the claim to every finite sequence. Matrix multiplication is associative and generally noncommutative. A wholly zero composite is rejected as having no valid initial state.

Interface composition processes each input against accumulated outputs. Compatible outputs bind internally. A required input with no preceding output becomes an external input. A same-name, incompatible output raises an error. Accumulated outputs persist, with latest-name overwrite. The composite stores remaining external inputs and all retained outputs. This is a contract abstraction: actual output values are not generated by the specification simulator.

Time, money, resource cost and energy sum for serial execution. Reliability multiplies under independent success assumptions. Aggregate risk is 1-product(1-risk_i), also assuming independent adverse events. Availability is logical AND and resource requirements are union. Correlated failures, consumable capacities and time-varying availability are not modeled. Risk and reliability are separate estimates and need not be complements.

Execution of a composite uses its transition matrix, retained input contract and aggregated gates. Composition is therefore possible structurally even when current availability makes execution impossible. Encoded composites can be passed back to encode or nested in compose. Associativity is checked on the full vectors for the supplied chains; arbitrary conflicting external port contracts require a richer binding model.

7. Implementation

embedding.py contains Encoded, Embedding.encode, Embedding.compose, Embedding.compatibility, Embedding.execute, Embedding.goal_score and similarity. NumPy implements the transition vectors and dense products. make_dataset.py builds dataset.json deterministically. experiments.py creates measured JSON, ordered-pair CSV files and NPZ arrays. test_embedding.py contains five regression test methods.

encode(state,"state") creates a basis vector; encode(goal,"goal") creates a goal mask; encode(capability) creates the full numerical vector. encode(encoded_composite) validates the schema and returns it. similarity defaults to the functional matrix block for capabilities. State-to-goal comparison uses a dot product through

goal_score rather than comparing different entity kinds with cosine.

Run from the package directory: python -m pip install -r requirements.txt; python make_dataset.py; OPENBLAS_NUM_THREADS=1 python experiments.py; python -m unittest -v. The README has a working end-to-end API example. Runtime dependencies are Python and NumPy only. This simulates executable specifications; it does not call external API, GUI or database services.

Input domains are checked by finite membership, and declared port type labels are used in interface matching. Domain values must themselves be appropriate for their declared types. Undefined variables, unsupported predicates or effects outside the declared state universe fail rather than extrapolate. Cross-schema similarity and composition are rejected. No random training, learned parameters or train/test split is involved.

8. Experimental methodology

Three synthetic problems are specified: commerce (Order -> Payment -> Notification), document processing (Document -> Signature -> Archive), and data processing (Extract -> Transform -> Publish). Each contains five Boolean variables and quantity in {0,1,2}, giving 96 states, one initial state and one terminal goal. Each has nine capabilities, explicit typed ports, quality values, resources, mechanisms and a three-step composition.

The nine capabilities comprise three chain steps, a contradictory cancellation, database and GUI alternatives to the first step, an irrelevant log operation, an incompatible typed input and a quantity/authentication-constrained variant. Commerce CreateOrder has no additional state precondition or capability constraint, exactly matching the required positive/negative pattern. A request input is external and does not alter that pattern.

Experiment 1 classifies all 81 ordered pairs per problem, including the required CreateOrder -> MakePayment and CreateOrder -> CancelCart pattern (CancelCart requires Order=false). The independent oracle interprets JSON guards and assignments across all states and checks interfaces without transition-matrix multiplication. A fixed cosine >=0.5 heuristic is compared as an intentionally simple baseline; the threshold is not fitted.

Experiment 2 compares the three-step composite against sequential interpretation for every source state, checks nested associativity, measures distance from matrix addition and records goal satisfaction. Experiment 3 measures semantic cosine and full-vector distances between API, database and GUI alternatives. Experiment 4 checks the irrelevant operation and leave-one-out necessity on the fixed chain. Experiment 5 ranks first-step alternatives by U=Rel-money and changes resource/availability gates.

All atomics are also checked against the oracle for applicability over every state. Negative regression tests cover missing input, resource absence, unavailability, numeric constraints, empty sequences, conflicting ports, schema mismatch and reversed order. Dense timings use 100 atomic encodings and 30 three-step compositions, one BLAS thread, one process, with arithmetic mean wall time. They are illustrative measurements without confidence intervals.

The three domains share a deliberately similar causal skeleton. This tests consistency under renaming and modest guard variation, not statistical generalization. All enumerated states are evaluated, including states not reachable from the supplied initial state. Real datasets, stochastic effects, broader graph shapes and scalability stress tests remain future work.

9. Results

All five test methods passed. Exact compatibility agreed on 243/243 pairs (100%); the cosine heuristic agreed on 138/243 (56.79%). There were 0/2,592 atomic applicability disagreements and 0/288 three-step composite semantic disagreements. The high exact agreement is expected from the exact finite semantics and

establishes implementation consistency, not a learned prediction advantage.

The required positive pairs had state coverage 1 and compatible ports; the negative cancellation pairs had coverage 0. WrongInput had state coverage 1 but failed its typed port requirement. Thus state compatibility alone is insufficient. API/database and API/GUI functional cosine was 1 to floating-point tolerance, but full-vector Euclidean distances were 2.0013 and 2.0040.

All composites achieved their terminal goal from the supplied initial state and all nested full-vector associativity checks passed. Commerce constituent cosines were 0.2500, 0.3536 and 0.3536; document/data values were 0.2500, 0.2041 and 0.2041. The composite is related to but not identical to its parts. Matrix-sum error was 15.4919 for commerce and 12.6491 for the other domains.

The irrelevant capability scored 0 for the initially false terminal goal; removing any chain step also scored 0. Adding the irrelevant log step after the chain left goal satisfaction at 1, showing that goal satisfaction alone cannot penalize redundant execution. The extra step increases quality costs, so relevance and operational accounting should be considered together.

For all domains, U=Rel-money ranked DATABASE first (0.94), with API and GUI tied at 0.89. Removing Database excludes that implementation; marking the API unavailable excludes API. The chain totals were 300 ms, money 0.70, resource_cost 3, energy 0.30, reliability 0.970299 and risk 0.029701. These are aggregate specification values, not measured external-service outcomes.

Domain	Exact pairs	Cosine >=0.5	Encode ms	Compose ms
commerce	81/81	46/81	0.2102	0.1741
document	81/81	46/81	0.1745	0.1999
data	81/81	46/81	0.1820	0.1690

10. Analysis

Capability representation and identity: all nine atomic full vectors in each problem are distinct; mechanisms distinguish alternatives despite equal state behavior. Extensionally equivalent specifications with identical metadata intentionally share a vector. State relationship and constraints: truth-table rows encode exactly when the operation is allowed and preserve unaffected variables; quantity/authentication gates are exercised in negative tests.

Precondition-effect and input-output relationships: directional coverage distinguishes the required compatible and contradictory pairs; typed interface matching rejects the WrongInput case even when all poststates meet its guards. Cosine is useful for finding functionally interchangeable candidates, but its symmetry makes it unsuitable as a composability certificate.

Composition and goal relevance: exact products preserve intermediate applicability, order and final goals. Leave-one-out testing shows why upstream steps matter although they do not directly set the terminal goal. This is fixed-sequence attribution; it does not discover another sequence or decide relevance in every possible context.

Operational properties: static attributes are represented without distorting semantic cosine. Cost and reliability alter preference; resources and availability alter feasibility. U uses an illustrative coefficient of 1 utility unit per money unit. Changing that coefficient can change the winner; an application should specify utility weights and tie-breaking. Raw operational values are not commensurable for arbitrary full-vector cosine.

Consistency: results match across the three problems, with guard-dependent composite cosine differences. Efficiency: state encoding is O(N) in this dense implementation; goal encoding is O(N times predicate count); atomic operator creation is O(N^2 + N times guard/effect work). A dense product is O(N^3), and a length-m chain costs O(mN^3). Capability storage is O(N^2+2L+H+J). Deterministic sparse operators would have at most N nonzeros and permit O(mN) successor-map composition, a future optimization.

Each measured problem has N=96, L=6, H=9, J=3 and d=9,247, or 73,976 bytes per float64 capability vector. A finite state basis and operator can preserve these formal relationships exactly. This answers the primary question positively within a bounded deterministic universe, while the lack of compression prevents claiming a practical low-dimensional embedding.

11. Limitations

The state universe grows as the product of variable-domain sizes. Exact Cartesian enumeration is exponentially large in the number of Boolean variables. At 20 Boolean variables, N=1,048,576 and a single dense N^2 float64 matrix requires roughly 8.8 TB. This construction is an exact research baseline, not a production-scale deployment design.

Continuous numeric, structured, relational and unbounded domains need abstraction or a different operator representation. Discretization can lose soundness unless its semantics are justified. Current effects are constant assignments; arithmetic updates, conditional effects, nondeterminism, failures, retries, temporal policies and concurrency are outside the implementation. Mechanism vocabulary is closed; new entries require rebuilding the schema.

Typed ports are matched by exact names and type labels with finite domain inclusion. Aliases, schema subtyping, value-dependent output guarantees, external-input intersection, optional-input forwarding, capacity consumption and arbitrary nested port collisions need stronger contract modeling. The supplied domains avoid these ambiguous cases. Availability is a static snapshot; resource requirements are presence checks.

The experiments are small, synthetic and mostly isomorphic. Exact success follows from using the same intended finite semantics for design and oracle, although the algorithms differ. There is no real-service measurement, external benchmark, learned embedding baseline, threshold tuning, uncertainty estimation or out-of-distribution study. Timing numbers are environment-dependent and cannot establish scalability.

Reliability multiplication and aggregate risk require independence; common-mode failures invalidate those estimates. The initial-state goal score is a deterministic conditional indicator and should not be interpreted as an observed probability. The simulator does not validate that implementation mechanisms actually realize the declared effects.

12. Conclusion

The FSCO design embeds states as basis vectors, goals as satisfaction masks and capabilities as vectorized partial transition operators with interface and operational features. It provides an exact ordered composition operation by matrix multiplication. The experiments demonstrate distinctions between functional similarity, directional compatibility, implementation identity and execution feasibility.

The strongest supported claim is that a vector representation can preserve all modeled deterministic relationships in a fully enumerated finite application. The strongest limitation is state-space size. The next useful research step is a sparse or factorized operator representation, followed by heterogeneous formal datasets and contract-aware compressed approximations with explicit soundness checks. No planner redesign is required to investigate those representation questions.

References and reproducibility

[1] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. https://arxiv.org/abs/1301.3781

[2] Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP-IJCNLP, 3982-3992. https://aclanthology.org/D19-1410/

[3] Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., and Yakhnenko, O. (2013). Translating Embeddings for Modeling Multi-relational Data. NeurIPS 26. https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf

[4] PCCST503 Assignment 2. Design of a Vector Embedding for Capability Composition. Provided assignment brief, 13 pages.

Environment: {"python": "3.12.14", "numpy": "2.3.5", "platform": "Linux-6.18.44-x86_64-with-glibc2.39"}

All reported numbers originate in results/results.json; individual ordered pair outcomes are in results/*_pairs.csv. Actual numerical encodings are in results/*_vectors.npz. The source dataset is dataset.json and the implementation can regenerate every result.

Deliverable mapping: formal design is FORMAL_DESIGN.md and sections 4-6; implementation is embedding.py with experiments.py and tests; experimental dataset is dataset.json with its generator; technical report is this PDF and its editable Markdown source.